

Programmation fonctionnelle 2

Preuves et programmes

CM2 : Prop

Pierre Rousselin

Université Sorbonne Paris Nord

janvier 2026

Les sortes de Rocq

La tactique `exact`

La conjonction (« et ») en Rocq

La disjonction (« ou ») dans Prop

L'implication

Résumé des tactiques correspondant à la déduction naturelle

Typer les types

- ▶ En Rocq (contrairement à OCaml), les types sont des termes comme les autres, ils ont donc des ... types :
`Check true. (* true : bool *)`
`Check bool. (* bool : Set *)`
`Check Set. (* Set : Type *)`
- ▶ On appelle une sorte (en anglais *sort*) un type de type. Ici, on voit **Set**, sorte qui contient `bool`, mais aussi `nat`, `Z`, `nat -> bool`, ...
- ▶ puis **Type** qui est une sorte qui contient **Set**, mais aussi **Set -> Set**, ...
- ▶ Et la sorte de **Type** ?
`Check Type. (* Type : Type ... ??? *)`
- ▶ (Paradoxe de Girard) Il est incohérent que **Type** soit de type **Type**...
- ▶ En fait Rocq nous a menti sur les affichages :
`Set Printing Universes.`
`Check Type. (* Type@{DemoCM2.28} : Type@{DemoCM2.28+1} *)`
- ▶ il y a une hiérarchie « d'univers »

$$\text{Set} \in \text{Type}(1) \in \text{Type}(2) \in \dots$$

mais les détails sont hors programme de ce cours et toujours très mouvants...

La sorte Prop

- ▶ La sorte **Prop** est intuitivement le type des propositions, des propriétés logiques (et non calculatoires) de Rocq.
- ▶ `Check 1 + 1 = 2. (* 2 = 2 : Prop *)`
`Check true = false. (* true = false : Prop *)`
`Check Prop. (* Prop : Type *)`
- ▶ On appelle *proposition* un type dont la sorte est **Prop**.
- ▶ Une *preuve* de cette proposition est un terme (on dit aussi habitant) de ce type.

La sorte Prop

- ▶ La sorte **Prop** est intuitivement le type des propositions, des propriétés logiques (et non calculatoires) de Rocq.
- ▶ `Check 1 + 1 = 2. (* 2 = 2 : Prop *)`
`Check true = false. (* true = false : Prop *)`
`Check Prop. (* Prop : Type *)`
- ▶ On appelle *proposition* un type dont la sorte est **Prop**.
- ▶ Une *preuve* de cette proposition est un terme (on dit aussi *habitant*) de ce type.
- ▶ L'un des buts de ce cours est de vous faire toucher du doigt ces deux dernières phrases.

Les sortes de Rocq

La tactique **exact**

La conjonction (« et ») en Rocq

La disjonction (« ou ») dans Prop

L'implication

Résumé des tactiques correspondant à la déduction naturelle

La tactique `exact`

- ▶ Dans une preuve, la tactique `exact` prend un argument et dit à Rocq : « ce terme est un type de la proposition à prouver. » Rocq vérifie alors que c'est bien le cas.
- ▶ Exemple le plus bête du monde :

```
Lemma foo (A : Prop) (h : A) : A.  
Proof. exact h. Qed.
```

La tactique `exact`

- Dans une preuve, la tactique `exact` prend un argument et dit à Rocq : « ce terme est un type de la proposition à prouver. » Rocq vérifie alors que c'est bien le cas.

- Exemple le plus bête du monde :

```
Lemma foo (A : Prop) (h : A) : A.  
Proof. exact h. Qed.
```

- Autre exemple : `True` est une proposition inductive à un seul constructeur qui s'appelle `I`.

```
Print True. (* Inductive True : Prop := I. *)  
Lemma bar : True.  
Proof. exact I. Qed.
```


La tactique `exact`

- ▶ Dans une preuve, la tactique `exact` prend un argument et dit à Rocq : « ce terme est un type de la proposition à prouver. » Rocq vérifie alors que c'est bien le cas.

- ▶ Exemple le plus bête du monde :

```
Lemma foo (A : Prop) (h : A) : A.  
Proof. exact h. Qed.
```

- ▶ Autre exemple : `True` est une proposition inductive à un seul constructeur qui s'appelle `I`.

```
Print True. (* Inductive True : Prop := I. *)  
Lemma bar : True.  
Proof. exact I. Qed.
```

- ▶ Dernier exemple (équivalent à `foo`) sur lequel on reviendra :

```
Lemma foo' (A : Prop) : A -> A.  
Proof. exact (fun (h : A) => h). Qed.
```

Les sortes de Rocq

La tactique `exact`

La conjonction (« et ») en Rocq

La disjonction (« ou ») dans Prop

L'implication

Résumé des tactiques correspondant à la déduction naturelle

Conjonction (« et ») de deux propositions

- ▶ La conjonction n'est pas primitive en Rocq. C'est un type inductif.
- ▶ **Notation** `"A /\ B"` := `(and A B)`
Inductive `and (A B : Prop) : Prop` := `conj (hA : A) (hB : B)`.
- ▶ En français : Si A et B sont deux propositions, `A /\ B` est une proposition que je peux prouver en fournissant une preuve de A et une preuve de B au constructeur `conj`.

Prouver un « et » (avec conj)

- Première possibilité, j'utilise **exact** avec **conj**
- Premier exemple un peu idiot :

Lemma useConj (A B : **Prop**) (hA : A) (hB : B) : A /\ B.

Proof. **exact** (conj hA hB). **Qed.**

- À vous maintenant :

Lemma useConj2 (A : **Prop**) (hA : A) : A /\ A.

Prouver un « et » (avec conj)

- Première possibilité, j'utilise **exact** avec **conj**
- Premier exemple un peu idiot :

```
Lemma useConj (A B : Prop) (hA : A) (hB : B) : A /\ B.  
Proof. exact (conj hA hB). Qed.
```

- À vous maintenant :

```
Lemma useConj2 (A : Prop) (hA : A) : A /\ A.  
Proof. exact (conj hA hA). Qed.
```

- Un dernier pour la route :

```
Lemma useConj3 (A B : Prop) (hA : A) (hB : B) : (A /\ B) /\ A.
```

Prouver un « et » (avec conj)

- Première possibilité, j'utilise **exact** avec **conj**
- Premier exemple un peu idiot :

```
Lemma useConj (A B : Prop) (hA : A) (hB : B) : A /\ B.  
Proof. exact (conj hA hB). Qed.
```

- À vous maintenant :

```
Lemma useConj2 (A : Prop) (hA : A) : A /\ A.  
Proof. exact (conj hA hA). Qed.
```

- Un dernier pour la route :

```
Lemma useConj3 (A B : Prop) (hA : A) (hB : B) : (A /\ B) /\ A.  
Proof. exact (conj (conj hA hB) hA). Qed.
```

Prouver un « et » (avec **split**)

- ▶ Comme **and** a un seul constructeur (à deux arguments) il n'y a qu'une seule façon « naturelle » de prouver $A \wedge B$.
- ▶ La tactique **split** permet de dire à Rocq : « je vais d'abord prouver ce qui est à gauche (premier argument du constructeur), puis ce qui est à droite (deuxième argument du constructeur) », générant ici deux sous-buts.

Lemma plop (A B : **Prop**) (h : A) (h' : B) : A $\wedge$ B.

Proof.

split.

- (* preuve de A *)

exact h.

- (* preuve de B *)

exact h'.

Qed.

- ▶ Intérêt 1 : se concentrer sur chaque sous-preuve (en général bien plus longue que dans l'exemple).
- ▶ Intérêt 2 : permet de mettre sous le tapis la définition de **and** pour se concentrer sur l'essentiel qui est bien sûr : « pour prouver $A \wedge B$ je dois prouver A et prouver B. »

Utiliser un « et » (à la main avec `match`)

- ▶ Finalement, `and` est un type inductif comme les autres.
- ▶ **Lemma** `proj1` (`A B : Prop`) (`h : A /\ B`) : `A`.
Proof.
 `exact (match h with conj hA _ => hA end).`
Qed.
- ▶ Idem avec un motif irréfutable (sucre syntaxique possible car `and` qui a un seul constructeur) :

Lemma `proj1'` (`A B : Prop`) (`h : A /\ B`) : `A`.

Proof.

`exact (let '(conj hA _) := h in hA).`

Qed.

- ▶ À vous pour :

Lemma `and_comm` (`A B : Prop`) (`h : A /\ B`) : `B /\ A`.

Proof.

`exact (let '(conj hA hB) := h in conj hB hA).`

Qed.

Utiliser un « et » avec `destruct`

- ▶ Écrire les termes de preuve à la main est un plaisir de fin gourmet...
- ▶ ... mais peut vite devenir usant.
- ▶ La tactique `destruct` permet de ne pas faire ce `match` à la main.

```
Lemma proj1'' (A B : Prop) (h : A /\ B) : A.
```

```
Proof.
```

```
  destruct h as [hA _]. exact hA.
```

```
Qed.
```

- ▶ Rappel : on écrit `destruct h as [var1 var2]` car `h` est de type `(and A B)`, qui a un seul constructeur à deux arguments.
- ▶ À vous pour :

```
Lemma and_comm (A B : Prop) (h : A /\ B) : B /\ A.
```

Utiliser un « et » avec `destruct`

- Écrire les termes de preuve à la main est un plaisir de fin gourmet...
- ... mais peut vite devenir usant.
- La tactique `destruct` permet de ne pas faire ce `match` à la main.

```
Lemma proj1'' (A B : Prop) (h : A /\ B) : A.
```

```
Proof.
```

```
  destruct h as [hA _]. exact hA.
```

```
Qed.
```

- Rappel : on écrit `destruct h as [var1 var2]` car `h` est de type `(and A B)`, qui a un seul constructeur à deux arguments.
- À vous pour :

```
Lemma and_comm (A B : Prop) (h : A /\ B) : B /\ A.
```

```
Proof.
```

```
  destruct h as [hA hB]. split.
```

```
  - exact hB.
```

```
  - exact hA.
```

```
Qed.
```

destruct imbriqués

- ▶ À vous pour :

```
Lemma and_assoc (A B C : Prop) (h : A /\ (B /\ C)) :  
  (A /\ B) /\ C.
```

destruct imbriqués

- ▶ À vous pour :

```
Lemma and_assoc (A B C : Prop) (h : A /\ (B /\ C)) :  
  (A /\ B) /\ C.
```

Proof.

```
destruct h as [hA hBC].  
destruct hBC as [hB hC].  
exact (conj (conj hA hB) hC).
```

Qed.

- ▶ Évidemment, on n'a pas envie de taper ce deuxième **destruct**.
On peut dans la partie entre crochets du premier **destruct**
continuer notre travail de destruction et imbriquer des crochets
dans des crochets :

```
Lemma and_assoc' (A B C : Prop) (h : A /\ (B /\ C)) :  
  (A /\ B) /\ C.
```

Proof.

```
destruct h as [hA [hB hC]].  
exact (conj (conj hA hB) hC).
```

Qed.

Règles de déduction naturelle

- ▶ Ce qui est intéressant (et intrigant) est que **split** et **destruct h as [h1 h2]** se comportent exactement comme les règles d'introduction et d'élimination du « et » en déduction naturelle
- ▶ Ces règles ont été énoncées par Gerhard Gentzen, mathématicien et logicien nazi (mort en 1945 dans un camp soviétique).

Élimination du « et »

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge\text{E-g})$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (\wedge\text{E-d})$$

- ▶ Le symbole $\vdash$ (qu'on lit en français « taquet ») signifie que ce qui est à droite peut se déduire de ce qui est à gauche.
- ▶ Ici, Γ désigne un *contexte* quelconque, un ensemble de déclarations ou de définitions.
- ▶ Au dessus de la barre verticale, on trouve les prémisses, et en dessous la conclusion.
- ▶ De haut en bas : si dans le contexte Γ je sais prouver $A \wedge B$, alors dans le contexte Γ , je sais prouver A et je sais prouver B (oui, ça a l'air un peu bête ce qu'on raconte).
- ▶ Équivalent à **destruct** h **as** [hA hB]

Introduction du « et »

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge I)$$

- ▶ De haut en bas : si dans le contexte Γ je sais prouver A et B , alors dans le contexte Γ , je sais prouver $A \wedge B$.
- ▶ De bas en haut, pour prouver $A \wedge B$ à partir de Γ il suffit de prouver A à partir de Γ et B à partir de Γ .
- ▶ Équivalent à la tactique **split** en Rocq.

Les sortes de Rocq

La tactique `exact`

La conjonction (« et ») en Rocq

La disjonction (« ou ») dans `Prop`

L'implication

Résumé des tactiques correspondant à la déduction naturelle

Disjonction (« ou » logique) dans Prop

- ▶ Pas de suspense, c'est encore non primitif en Rocq.
- ▶ **Inductive** `or (A B : Prop) : Prop :=`
 `or_introl (hA : A) | or_intror (hB : B).`
Notation `"A \/\ B"` := `(or A B).`
- ▶ Cette fois, (admirez la dualité), deux constructeurs, chacun a un seul argument.
- ▶ En français, une preuve de `A \/\ B` est
 - ▶ construite avec le constructeur `or_introl` et une preuve de `A`, **ou**
 - ▶ construite avec le constructeur `or_intror` et une preuve de `B`.
- ▶ Si `A` et `B` sont fixés et de type **Prop**, alors
 - ▶ `or_introl : A -> A \/\ B`
 - ▶ `or_intror : B -> A \/\ B`
- ▶ en vrai, il y a une subtilité, car `A` et `B` sont eux-mêmes paramètres (implicites) de `or_introl` et `or_intror`

Prouver un « ou » à la main avec **exact** et les constructeurs de **or**

- ▶ **Lemma** foo1 (A B : **Prop**) (hA : A) : A **\|** B.
 Proof. **exact** (or_introl hA). **Qed.**
- ▶ **Lemma** foo2 (A B : **Prop**) (hB : B) : A **\|** B.

Prouver un « ou » à la main avec `exact` et les constructeurs de `or`

- ▶ `Lemma foo1 (A B : Prop) (hA : A) : A \/ B.`
`Proof. exact (or_introl hA). Qed.`
- ▶ `Lemma foo2 (A B : Prop) (hB : B) : A \/ B.`
`Proof. exact (or_intror hB). Qed.`
- ▶ `Lemma foo4 (A B : Prop) (h : A /\ B) : A \/ B.`

Prouver un « ou » à la main avec `exact` et les constructeurs de `or`

- ▶ `Lemma foo1 (A B : Prop) (hA : A) : A \/ B.`
`Proof. exact (or_introl hA). Qed.`
- ▶ `Lemma foo2 (A B : Prop) (hB : B) : A \/ B.`
`Proof. exact (or_intror hB). Qed.`
- ▶ `Lemma foo4 (A B : Prop) (h : A /\ B) : A \/ B.`
`Proof. exact (let '(conj _ hB) := h in or_intror hB). Qed.`

Prouver un « ou » avec **left** ou **right**

- Pour prouver un but (une proposition) qui a deux constructeurs, on peut utiliser la tactique **left** pour dire qu'on a l'intention de prouver le but (c'est-à-dire construire un terme dont le type est le but) avec le premier de ces deux constructeurs,
- ou plutôt, utiliser la tactique **right** pour prouver le but avec le deuxième constructeur.

Lemma foo1 (A B : **Prop**) (hA : A) : A $\vee$ B.

Proof. **left.** **exact** hA. **Qed.**

- **Lemma** foo2 (A B : **Prop**) (hB : B) : A $\vee$ B.

Prouver un « ou » avec **left** ou **right**

- Pour prouver un but (une proposition) qui a deux constructeurs, on peut utiliser la tactique **left** pour dire qu'on a l'intention de prouver le but (c'est-à-dire construire un terme dont le type est le but) avec le premier de ces deux constructeurs,
- ou plutôt, utiliser la tactique **right** pour prouver le but avec le deuxième constructeur.

```
Lemma foo1 (A B : Prop) (hA : A) : A  $\vee$  B.
```

```
Proof. left. exact hA. Qed.
```

- Lemma foo2 (A B : Prop) (hB : B) : A $\vee$ B.

```
Proof. right. exact hB. Qed.
```

- Lemma foo4 (A B : Prop) (h : A $\wedge$ B) : A $\vee$ B.

Prouver un « ou » avec **left** ou **right**

- Pour prouver un but (une proposition) qui a deux constructeurs, on peut utiliser la tactique **left** pour dire qu'on a l'intention de prouver le but (c'est-à-dire construire un terme dont le type est le but) avec le premier de ces deux constructeurs,
- ou plutôt, utiliser la tactique **right** pour prouver le but avec le deuxième constructeur.

```
Lemma foo1 (A B : Prop) (hA : A) : A  $\vee$  B.
```

```
Proof. left. exact hA. Qed.
```

- Lemma foo2 (A B : Prop) (hB : B) : A $\vee$ B.

```
Proof. right. exact hB. Qed.
```

- Lemma foo4 (A B : Prop) (h : A $\wedge$ B) : A $\vee$ B.

```
Proof. destruct h as [hA _]. left. exact hA. Qed.
```

Utiliser un « ou » à la main avec `match`

- ▶ `Lemma bar1 (A : Prop) (H : A \/\ A) : A.`
`Proof.`
 `exact (`
 `match H with`
 `| or_introl ha => ha`
 `| or_intror ha => ha`
 `end).`
`Qed.`
- ▶ `Lemma bar2 (A B : Prop) (H : A \/\ B) : B \/\ A.`

Utiliser un « ou » à la main avec `match`

► `Lemma bar1 (A : Prop) (H : A  $\vee$  A) : A.`

`Proof.`

```
  exact (
    match H with
    | or_introl ha => ha
    | or_intror ha => ha
  end).
```

`Qed.`

► `Lemma bar2 (A B : Prop) (H : A  $\vee$  B) : B  $\vee$  A.`

`Proof.`

```
  exact (
    match H with
    | or_introl hA => or_intror hA
    | or_intror hB => or_introl hB
  end).
```

`Qed.`

Utiliser un « ou » avec `destruct`

- ▶ Là aussi, `exact` et `match` suffisent pour les cas très simples, mais en général on utilise `destruct`.
- ▶ On rappelle que `or A B` a deux constructeurs qui ont chacun un argument (une preuve de `A` pour le premier, une preuve de `B` pour le second).
- ▶ Donc on détruit (raisonne par cas sur) une preuve de $h : A \vee B$ avec
`destruct h as [var1 | var2]`.

Utiliser un « ou » avec destruct

- ▶ `Lemma bar1' (A : Prop) (H : A \ / A) : A.`
`Proof.`
 `destruct H as [hA | hA].`
 - `exact hA.`
 - `exact hA.`
`Qed.`
`Lemma bar1'' (A : Prop) (H : A \ / A) : A.`
`Proof.`
 `destruct H as [hA | hA]; exact hA.`
`Qed.`
- ▶ `Lemma bar2' (A B : Prop) (H : A \ / B) : B \ / A.`

Utiliser un « ou » avec destruct

- `Lemma bar1' (A : Prop) (H : A \ / A) : A.`
`Proof.`
 `destruct H as [hA | hA].`
 - `exact hA.`
 - `exact hA.`
`Qed.`
`Lemma bar1'' (A : Prop) (H : A \ / A) : A.`
`Proof.`
 `destruct H as [hA | hA]; exact hA.`
`Qed.`
- `Lemma bar2' (A B : Prop) (H : A \ / B) : B \ / A.`
`Proof.`
 `destruct H as [hA | hB].`
 - `right. exact hA.`
 - `left. exact hB.`
`Qed.`

Introductions du « ou »

Prouver avec les tactiques **left** et **right** revient à appliquer les deux règles d'introduction du « ou » en déduction naturelle.

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \text{ (}\forall\text{I-g)}$$

$$\frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} \text{ (}\forall\text{I-d)}$$

- ▶ De haut en bas : si dans le contexte Γ je sais prouver A ou, si dans le contexte Γ , je sais prouver B , alors dans le contexte Γ , je sais prouver $A \vee B$.
- ▶ De bas en haut, pour prouver $A \vee B$ à partir de Γ il suffit de prouver A à partir de Γ ou B à partir de Γ .

Élimination du « ou »

Utiliser une preuve de « ou » avec **destruct** revient à appliquer la règle d'élimination du « ou » en déduction naturelle.

$$\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash P \quad \Gamma, B \vdash P}{\Gamma \vdash P} (\vee E)$$

- ▶ De haut en bas : si dans le contexte Γ , je sais prouver $A \vee B$, et dans le contexte Γ, A je sais prouver P , et dans le contexte Γ, B , je sais aussi prouver P , alors dans le contexte Γ , je sais prouver P .
- ▶ De bas en haut, pour prouver P à partir du contexte Γ , il suffit de prouver $A \vee B$ à partir de Γ , puis il suffit de prouver P à partir de Γ, A et P à partir de Γ, B , autrement dit de se placer dans le cas où A est vrai puis dans le cas où B est vrai.

Les sortes de Rocq

La tactique `exact`

La conjonction (« et ») en Rocq

La disjonction (« ou ») dans Prop

L'implication

Résumé des tactiques correspondant à la déduction naturelle

L'implication est le type des fonctions

- ▶ Soient A et B de type **Prop**.
- ▶ Vue comme une fonction, un terme (une preuve) h de type $A \rightarrow B$ associe à un terme (une preuve) hA de type A un terme (une preuve) $(h\ hA)$ de type B .
- ▶ Autrement dit, si j'ai :
 - ▶ $hA : A$
 - ▶ $h : A \rightarrow B$Alors $(h\ hA)$ est une preuve de B .
- ▶ C'est juste le *modus ponens* habituel.
- ▶ C'est aussi juste la règle de typage habituelle de l'application des fonctions.

Utiliser une implication en composant directement les fonctions

► **Lemma** `modus_ponens` $(A\ B : \text{Prop})\ (hA : A)\ (h : A \rightarrow B) : B.$

Utiliser une implication en composant directement les fonctions

```
► Lemma modus_ponens (A B : Prop) (hA : A) (h : A -> B) : B.  
Proof.  
  exact (h hA).  
Qed.
```

Utiliser une implication en composant directement les fonctions

- ▶ `Lemma modus_ponens (A B : Prop) (hA : A) (h : A -> B) : B.`
`Proof.`
`exact (h hA).`
`Qed.`
- ▶ `Lemma impl_trans1 (A B C : Prop) (h1 : A -> B) (h2 : B -> C)`
`(hA : A) : C.`

Utiliser une implication en composant directement les fonctions

- ▶ `Lemma modus_ponens (A B : Prop) (hA : A) (h : A -> B) : B.`
`Proof.`
`exact (h hA).`
`Qed.`
- ▶ `Lemma impl_trans1 (A B C : Prop) (h1 : A -> B) (h2 : B -> C)`
`(hA : A) : C.`
`Proof.`
`exact (h2 (h1 hA)).`
`Qed.`

Utiliser une implication avec `apply`

- ▶ Si le but est une proposition B
- ▶ Et que h est une preuve (un terme) de $A \rightarrow B$.
- ▶ Alors la tactique `apply h` :
 - ▶ compare la conclusion de l'implication (le type de retour de la fonction) h avec le but.
 - ▶ Si c'est la même chose¹,
 - ▶ Rocq remplace le but par la prémisse (ou le domaine) de l'implication, ici A .
- ▶ **Lemma** `modus_ponens` ' ($A \ B : \text{Prop}$) ($hA : A$) ($h : A \rightarrow B$) : B .

1. éventuellement avec unification, on verra ça dans un autre cours

Utiliser une implication avec `apply`

- ▶ Si le but est une proposition `B`
- ▶ Et que `h` est une preuve (un terme) de `A -> B`.
- ▶ Alors la tactique `apply h` :
 - ▶ compare la conclusion de l'implication (le type de retour de la fonction) `h` avec le but.
 - ▶ Si c'est la même chose¹,
 - ▶ Rocq remplace le but par la prémisse (ou le domaine) de l'implication, ici `A`.
- ▶ **Lemma** `modus_ponens` ' (`A B : Prop`) (`hA : A`) (`h : A -> B`) : `B`.
Proof.
 `apply h.` (** le but devient A **)
 `exact hA.`
Qed.

1. éventuellement avec unification, on verra ça dans un autre cours

Utiliser une implication avec `apply`

- ▶ Si le but est une proposition `B`
- ▶ Et que `h` est une preuve (un terme) de `A -> B`.
- ▶ Alors la tactique `apply h.` :
 - ▶ compare la conclusion de l'implication (le type de retour de la fonction) `h` avec le but.
 - ▶ Si c'est la même chose¹,
 - ▶ Rocq remplace le but par la prémisse (ou le domaine) de l'implication, ici `A`.
- ▶ `Lemma modus_ponens' (A B : Prop) (hA : A) (h : A -> B) : B.`
`Proof.`
`apply h. (* le but devient A *)`
`exact hA.`
`Qed.`
- ▶ `Lemma impl_trans1' (A B C : Prop) (h1 : A -> B) (h2 : B -> C)`
`(hA : A) : C.`

1. éventuellement avec unification, on verra ça dans un autre cours

Utiliser une implication avec `apply`

- ▶ Si le but est une proposition B
- ▶ Et que `h` est une preuve (un terme) de $A \rightarrow B$.
- ▶ Alors la tactique `apply h.` :
 - ▶ compare la conclusion de l'implication (le type de retour de la fonction) `h` avec le but.
 - ▶ Si c'est la même chose¹,
 - ▶ Rocq remplace le but par la prémisse (ou le domaine) de l'implication, ici `A`.
- ▶ `Lemma modus_ponens' (A B : Prop) (hA : A) (h : A → B) : B.`
`Proof.`
`apply h. (* le but devient A *)`
`exact hA.`
`Qed.`
- ▶ `Lemma impl_trans1' (A B C : Prop) (h1 : A → B) (h2 : B → C)`
`(hA : A) : C.`
- ▶ `Proof.`
`apply h2. (* le but devient B *)`
`apply h1. (* le but devient A *)`
`exact hA.`
`Qed.`

1. éventuellement avec unification, on verra ça dans un autre cours

Élimination de $\rightarrow$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash A \rightarrow B}{\Gamma \vdash B} (\rightarrow E)$$

- ▶ De haut en bas : si dans le contexte Γ je sais prouver A et $A \rightarrow B$, alors dans le contexte Γ , je sais prouver B .
- ▶ De bas en haut, pour prouver B partir de Γ il suffit de prouver A et $A \rightarrow B$ à partir de Γ .

Attention au **apply** non réfléchi

Lemma plop (P Q R : **Prop**) (h1 : P) (h2 : P -> R)
(h3 : Q -> R) : R.

Proof.

apply h3.

Qu'en pensez-vous ?

- ▶ Attention, **apply** une implication est un choix.
- ▶ Il se peut que ça ne soit pas le bon.
- ▶ Lorsqu'on est en « mode jeu vidéo » dans les preuves en Rocq, on a tendance à avoir des œillères et on a **du mal à revenir en arrière**.
- ▶ Il faut regarder attentivement l'état de la preuve, réfléchir à la prouvabilité du but dans tous les cas et savoir dire (mais c'est dur pour les humains), « j'ai fait le mauvais choix. »
- ▶ C'est aussi valable pour les tactiques **left** et **right**.

Prouver une implication à la main avec une fonction anonyme

- ▶ Rappel : la fonction qui à x associe $x + 1$ (par exemple) est notée :
 - ▶ `fun n -> n + 1` en OCaml
 - ▶ `fun n => n + 1` en Rocq
- ▶ Sachant qu'une implication n'est qu'un type de fonctions, il suffit pour prouver $A \rightarrow B$ de fournir une fonction qui prend un argument de type A et fournit une valeur de type B .
- ▶ `Lemma impl_refl (A : Prop) : A -> A.`
`Proof.`
`exact (fun h => h).`
`Qed.`

Prouver une implication à la main avec une fonction anonyme

- ▶ Rappel : la fonction qui à x associe $x + 1$ (par exemple) est notée :
 - ▶ `fun n -> n + 1` en OCaml
 - ▶ `fun n => n + 1` en Rocq
- ▶ Sachant qu'une implication n'est qu'un type de fonctions, il suffit pour prouver $A \rightarrow B$ de fournir une fonction qui prend un argument de type A et fournit une valeur de type B .
- ▶ `Lemma impl_refl (A : Prop) : A -> A.`
`Proof.`
`exact (fun h => h).`
`Qed.`
- ▶ `Lemma impl_trans (A B C : Prop) (h1 : A -> B) (h2 : B -> C) :`
`A -> C.`
`Proof.`

Prouver une implication à la main avec une fonction anonyme

- ▶ Rappel : la fonction qui à x associe $x + 1$ (par exemple) est notée :
 - ▶ `fun n -> n + 1` en OCaml
 - ▶ `fun n => n + 1` en Rocq
- ▶ Sachant qu'une implication n'est qu'un type de fonctions, il suffit pour prouver $A \rightarrow B$ de fournir une fonction qui prend un argument de type A et fournit une valeur de type B .
- ▶ `Lemma impl_refl (A : Prop) : A -> A.`
`Proof.`
`exact (fun h => h).`
`Qed.`
- ▶ `Lemma impl_trans (A B C : Prop) (h1 : A -> B) (h2 : B -> C) :`
`A -> C.`
`Proof.`
`exact (fun hA => (h2 (h1 hA))).`
`Qed.`

Prouver une implication avec la tactique `intros`

- ▶ Encore une fois c'est plaisant et efficace sur des preuves courtes d'écrire le terme de preuve directement.
- ▶ Mais on se voit mal commencer une preuve de 100 lignes par :

```
exact (fun x1 x2 x3 =>  
  (* 100 lignes plus loin *)  
).
```
- ▶ La tactique `intros` permet de nommer ces nouvelles variables `x1`, `x2` et `x3` et de continuer la preuve avec ces variables dans le contexte local de la preuve.

Prouver une implication avec la tactique intros

► `Lemma impl_refl' (A : Prop) : A -> A.`

`Proof.`

`intros h. (* le but devient A
 et on a (h : A) dans le contexte *)`

`exact h.`

`Qed.`

Prouver une implication avec la tactique intros

► `Lemma impl_refl' (A : Prop) : A -> A.`

`Proof.`

`intros h. (* le but devient A
 et on a (h : A) dans le contexte *)`

`exact h.`

`Qed.`

► `Lemma impl_trans' (A B C : Prop) (h1 : A -> B) (h2 : B -> C) :
 A -> C.`

`Proof.`

Prouver une implication avec la tactique intros

► **Lemma** impl_refl' (A : **Prop**) : A -> A.

Proof.

intros h. *(* le but devient A
 et on a (h : A) dans le contexte *)*

exact h.

Qed.

► **Lemma** impl_trans' (A B C : **Prop**) (h1 : A -> B) (h2 : B -> C) :
 A -> C.

Proof.

intros hA. *(* le but devient C
 et on a (hA : A) dans le contexte *)*

apply h2.

apply h1.

exact hA.

Qed.

Règle d'introduction de $\rightarrow$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} (\rightarrow I)$$

- ▶ De haut en bas : si dans le contexte Γ avec en plus l'hypothèse que A est vrai (et aucune autre hypothèse supplémentaire), je sais prouver B ; alors dans le contexte Γ , je sais prouver $A \rightarrow B$.
- ▶ De bas en haut, pour prouver à partir de Γ la formule $A \rightarrow B$, il suffit de prouver B à partir de Γ auquel on ajoute l'hypothèse que A est vraie (et aucune autre hypothèse).

Les sortes de Rocq

La tactique `exact`

La conjonction (« et ») en Rocq

La disjonction (« ou ») dans Prop

L'implication

Résumé des tactiques correspondant à la déduction naturelle

Tactiques et règles de déduction naturelle

On a déjà passé en revue presque toutes les tactique correspondant à des règles de déduction naturelle en Rocq.

connecteur	introduction (prouver)	élimination (utiliser)
$\rightarrow$	intros	apply
$\wedge$	split	destruct
$\vee$	left ou right	destruct
$\forall$	intros	specialize , instantiation ²
$\perp$		destruct
$\exists$	exists	destruct

Dans chaque cas, il s'agit, en fait :

- ▶ de définition (**intros**) de fonctions ou
- ▶ d'application (**apply**) de fonctions ou
- ▶ d'élimination (**destruct**) d'un terme dont le type est inductif (donc en fait **match**) ou
- ▶ de construction (**split**, **left**, **right**) d'un élément d'un type inductif avec un de ses constructeurs appliqué à des arguments du bon type

2. explicite ou implicite avec unification